Manuel HTML5 & CSS3 – Guide complet du développeur web

Manuel Regamey – 2025

HTML5-CSS3

Partie I : Introduction au Web et au HTML

Liens

- Différence **Internet / Web**
- Les URL : schéma, domaine, chemin, requête, fragment
- Protocole **HTTP / HTTPS** : rôle, codes de statut, stateless
- Hypertexte et liens `<a>`
- Qu'est-ce que HTML ? Un langage de balisage sémantique
- Différence **HTML / CSS / JavaScript** et séparation des rôles
- Bref historique : HTML → XHTML → HTML5 → *Living Standard*
- Squelette minimal d'une page HTML5 : `<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`
- Pièges fréquents : encodage, doctype, balises mal fermées

Réf. MDN – HTML élément

Partie II : Les bases du HTML

Liens

- Structure d'un document HTML5
- Balises de contenu texte : `<h1>` à `<h6>`, `<p>`, `<br>`, `<hr>`
- Mise en valeur : `<strong>`, `<em>`, `<mark>`, `<small>`, `<del>`, `<ins>`
- Listes : `<ul>`, `<ol>`, `<li>`
- Liens hypertextes : `<a href="...">`, attributs `target`, `title`

- Images : `<img src="..." alt="...">`, rôle de `alt` pour l'accessibilité
- Bonnes pratiques : titres hiérarchisés, balises sémantiques

Réf. MDN – Balises HTML

Partie III : Structure sémantique et organisation

Liens

- Différence `<div>` vs balises sémantiques HTML5 : `<header>`, `<footer>`, `<main>`, `<nav>`, `<article>`, `<section>`, `<aside>`
- Importance pour l'accessibilité (landmarks ARIA implicites)
- Impact sur le SEO et la lisibilité du code
- Hiérarchie des titres (`h1`, `h2`, `h3`…) et règles d'imbrication
- Bonnes pratiques : un seul `<main>`, sections toujours titrées

Réf. MDN – HTML sémantique

Partie IV : Métadonnées et ressources externes

Liens

- Le `<head>` et son rôle
- Métadonnées essentielles :
- `<meta charset="utf-8">` (encodage)
- `<meta name="viewport">` (affichage mobile)
- `<title>` unique et descriptif
- `<meta name="description">` (SEO)
- Lier des ressources avec `<link>` : CSS, favicon, manifest
- Charger du JavaScript avec `<script>` : `defer`, `async`, `type="module"`
- Ordre recommandé dans `<head>`

Réf. MDN – `<head>`

Partie V : Introduction au CSS

Liens

- Rôle du CSS par rapport au HTML

- Trois manières d'ajouter du style :
- inline → à éviter,
- interne (`<style>`),
- externe (`.css`) → bonne pratique
- Sélecteurs : balise, `.classe`, `#id`, groupement, descendant
- Propriétés essentielles : `color`, `background`, `font-family`, `font-size`, `line-height`, `text-align`, `margin`, `padding`, `border`
- Bonnes pratiques : unités relatives (`rem`, `em`), lisibilité, séparation des fichiers

Réf. MDN – CSS sélecteurs

Partie VI : Mise en page moderne (HTML + CSS)

Liens

- Le **box model** : content → padding → border → margin
- `box-sizing: border-box` et marge fusionnée (margin collapsing)
- `display: block`, `inline`, `inline-block`
- Flexbox : axes, alignement, `gap`, propriétés `flex` (`grow`, `shrink`, `basis`)
- CSS Grid : colonnes, lignes, unités `fr`, fonctions `minmax()`, `auto-fit`
- Responsive design avec media queries

Réf. MDN – Flexbox

Partie VII : Médias et formulaires

Liens

- Médias : `<audio>`, `<video>`, `<iframe>`
- Formats multiples, attributs `controls`, `preload`, `poster`, `track` (sous-titres)
- Sécurité des iframes : `title`, `loading="lazy"`, `sandbox`, `referrerpolicy`
- Formulaires : `<form>`, `action`, `method`, `enctype`
- Champs `<input>` et variantes : texte, email, url, tel, number, range, date/time, color, file, checkbox, radio
- Autres contrôles : `<textarea>`, `<select>`, `<button>`
- Validation native : `required`, `pattern`, `min`, `max`

- Accessibilité : labels, fieldset/legend, aria-describedby

Réf. MDN – Formulaires

Partie VIII : Bonnes pratiques et outils

Liens

- Validation du code : W3C Validator (HTML), CSS Validator
- Accessibilité : `alt`, `label`, titres hiérarchisés, contraste, navigation clavier
- Éviter le code obsolète : `<font>`, `<center>`, styles inline, JS inline
- Séparation contenu / style / script
- Lisibilité : indentation, noms de classes explicites, structure de projet claire
- Outils développeur intégrés (Inspecteur, Console, Réseau, Lighthouse)
- Documentation fiable : MDN, WHATWG Living Standard, W3C, Can I use, Web.dev

Réf. MDN – Accessibilité

I Introduction au Web et au HTML

I-1 Objectifs pédagogiques

Cette première séance a pour but de faire un rappel des bases. Il faut être capable de distinguer **Internet** du **Web**, d'identifier les éléments d'une **URL**, de comprendre ce que fait le protocole **HTTP/HTTPS** et pourquoi on parle de **langage de balisage sémantique** quand on décrit le HTML. Nous apprendrons aussi à différencier clairement les rôles respectifs de **HTML**, **CSS** et **JavaScript**, et nous écrirons une première page HTML valide.

I-2 Pré-requis et environnement

Nous n'avons pas besoin d'un équipement complexe pour démarrer : un **navigateur récent** (Firefox, Chrome, Edge…) et un **éditeur de texte** ou un **éditeur de code** suffisent. Dans la formation, nous utilisons le plus souvent Visual Studio Code, mais nous pourrions également travailler avec `Notepad++` (sous Windows) ou, en ligne de commande, avec `nano` ou `vim` (sous Linux).

À ce stade, aucune installation serveur n'est nécessaire. En revanche, au lieu d'ouvrir directement nos fichiers HTML avec le protocole `file://`, nous préférerons utiliser **Live Server** dans VS Code. Ce module lance un petit serveur local (`http://localhost`), ce qui nous évite les limitations de sécurité du navigateur (notamment liées au **CORS**, aux modules JavaScript, ou aux requêtes fetch), et nous permet de travailler dans des conditions proches d'un vrai site web.

I-3 Internet, Web et hypertexte

Il est essentiel de comprendre ce qu'est le Web et ce qui le distingue d'Internet.

Internet vs Web

Internet est un réseau mondial d'ordinateurs interconnectés. Il transporte des données, un peu comme un gigantesque système d'autoroutes reliant des villes. Le **Web**, ou **World Wide Web**, est un service qui circule sur cette infrastructure. Il repose sur l'**hypertexte** (la possibilité de relier des documents), les **URL** (adresses des ressources) et le protocole **HTTP/HTTPS**.

En résumé : Internet est le réseau, le Web est une application qui l'utilise.

L'URL

Une **URL** (*Uniform Resource Locator*) est l'adresse d'une ressource sur le Web. Par exemple :

```
https://www.exemple.org:443/articles/42?tri=recent#comment-plouf
```

Cette adresse contient plusieurs morceaux : le **schéma** (`https`), le **nom de domaine** (`www.exemple.org`), parfois un **port** (`443`), un **chemin** vers une ressource (`/articles/42`), une éventuelle **chaîne de requête** (`?tri=recent`) et un **fragment** (`#comment-plouf`). Une URL peut être **absolue** (toutes les informations sont présentes) ou **relative** (sa résolution dépend du document courant). Pour plus de détails, voir la page URL sur MDN.

HTTP et HTTPS

Le **protocole HTTP** organise la communication entre ton navigateur (le **client**) et le serveur web. Le client envoie une requête, et le serveur répond avec un **code de statut** (succès, erreur, redirection…), des **en-têtes** qui précisent des informations techniques, et enfin le contenu demandé (HTML, image, JSON, etc.). La version sécurisée **HTTPS** chiffre les échanges via TLS, garantissant que personne ne puisse lire ou modifier les données en transit. Plus d'infos : HTTP - MDN.

Un point important : HTTP est **stateless**. Il ne se « souvient » pas des requêtes précédentes. Pour garder une session ouverte (par exemple pour rester connecté sur un site), on utilise des cookies ou des tokens.

L'hypertexte

Le concept d'**hypertexte** est ce qui fait la force du Web : des documents reliés entre eux par des liens. En HTML, on crée un lien avec la balise `<a>`. En cliquant de page en page, on « tisse » la toile (web).

I-4 HTML : un langage de balisage sémantique

Le HTML (*HyperText Markup Language*) n'est pas un langage de programmation mais un **langage de balisage**. Cela signifie qu'il permet d'**enrichir un texte** avec des balises pour indiquer le rôle de chaque portion de contenu.

Les balises sont entourées de chevrons (`<  >`). La plupart vont par paire : une **ouvrante** et une **fermante**. Exemple :

```
<p>Bonjour</p>
```

Certaines balises sont dites *vides* car elles n'ont pas de contenu : `<br>` pour un saut de ligne, `<img>` pour une image, `<meta>` pour des métadonnées, etc.

La sémantique

La grande force du HTML5 est sa dimension **sémantique** : chaque balise donne du sens au contenu. Par exemple :

- `<h1>` marque un titre principal
- `<nav>` signale une zone de navigation
- `<article>` encadre un article complet
- `<em>` met un mot en emphase

Ce choix améliore l'**accessibilité** (lecteurs d'écran, navigation clavier), le **référencement** (SEO) et la **lisibilité du code** pour les développeurs. Voici la référence HTML complète sur MDN.

Syntaxe et bonnes pratiques

- Les noms de balises et d'attributs ne sont pas sensibles à la casse, mais la convention est de tout écrire en **minuscules**.
- Les attributs suivent la forme `nom="valeur"`. Certains sont booléens (la seule présence de l'attribut suffit) comme `checked`, `required` ou `disabled`.
- Des attributs **globaux** comme `id`, `class` ou `title` peuvent être appliqués à presque toutes les balises.
- L'imbrication doit être respectée : une balise ouverte doit être refermée avant de clore la balise qui l'entoure.

I-5 HTML, CSS et JavaScript

Ces trois technologies sont indissociables du Web, mais chacune a un rôle bien distinct :

- **HTML** : structure et sens du contenu (titres, paragraphes, images, formulaires…).
- **CSS** : présentation et mise en forme (couleurs, polices, marges, disposition responsive).
- **JavaScript** : comportement et interactions (menus dynamiques, formulaires interactifs, requêtes réseau).

Ce découpage est appelé **séparation des préoccupations**. Une bonne pratique est d'écrire d'abord une page qui reste utile en HTML seul, puis d'ajouter le style avec CSS, et enfin les comportements avec JavaScript. C'est ce qu'on appelle le **progressive enhancement**. Voir : HTML vs CSS vs JavaScript - MDN.

I-6 Bref historique d'HTML

- Début des années 1990 : Tim Berners-Lee crée les premières versions au CERN.

- 1995 : HTML 2.0 devient la première norme officielle.
- 1997-1999 : HTML 3.2 puis 4.01 introduisent de nombreuses nouveautés.
- 2000 : apparition de XHTML, une réécriture plus stricte.
- 2014 : HTML5 devient une recommandation officielle du W3C.
- Depuis 2019 : le **Living Standard** du WHATWG est la référence unique, mise à jour en continu.

Aujourd'hui, on parle encore souvent d'« HTML5 » pour simplifier, mais il faut garder en tête qu'il évolue sans cesse.

I-7 Squelette d'un document HTML

Un document HTML minimal contient quatre briques :

```
<!DOCTYPE html>
<html>
  <head> </head>
  <body></body>
</html>
```

En pratique, il est conseillé d'ajouter quelques éléments essentiels :

- `lang="fr"` pour indiquer la langue du document,
- `<meta charset="utf-8">` pour gérer correctement les accents,
- `<title>` pour nommer l'onglet.

Exemple :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <title>Ma première page</title>
  </head>
  <body>
    Contenu de ma première page.
  </body>
</html>
```

I-8 Pièges fréquents

Quelques erreurs classiques à éviter :

- Oublier `<!DOCTYPE html>` et déclencher le mode « quirks ».

- Ne pas préciser l'encodage UTF-8 et voir les caractères accentués s'afficher bizarrement.
- Fermer incorrectement les balises, ce qui casse la structure.
- Utiliser uniquement `<div>` sans profiter des balises sémantiques (`<header>`, `<main>`, `<article>`…).
- Confondre structure et présentation en mettant du style directement dans les balises au lieu d'utiliser une feuille CSS.

I-9 Fiche-mémo

- Une **URL** se compose d'un schéma, d'un domaine, d'un chemin, éventuellement d'une requête et d'un fragment.
- **HTTP/HTTPS** organise le dialogue client-serveur.
- **HTML** structure et sémantise le contenu.
- **CSS** gère la présentation.
- **JavaScript** gère l'interactivité.
- Une page valide doit commencer par `<!DOCTYPE html>`, préciser la langue et l'encodage, et contenir un `<title>`.

II Les bases du HTML

II-1 Objectifs pédagogiques

Dans cette deuxième séance, nous allons apprendre à écrire les **premiers contenus réels** dans une page HTML. Nous découvrirons comment structurer du texte avec des titres et des paragraphes, comment insérer des sauts de ligne et des séparations thématiques, et surtout comment mettre en valeur certaines portions de texte de façon **sémantique**. Nous verrons également comment créer des listes, insérer des liens hypertextes avec leurs attributs essentiels, et afficher des images en respectant les bonnes pratiques d'accessibilité et de performance.

II-2 Structure de base rappelée

Avant d'aller plus loin, reprenons le squelette d'une page HTML5 minimale, légèrement enrichi :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <title>Titre de la page</title>
    <meta name="viewport" content="width=device-width, initial-scale=1" />
  </head>
  <body>
    Contenu…
```

```
  </body>
</html>
```

Quelques rappels et points essentiels :

- `<!DOCTYPE html>` active le mode standard.
- L'attribut `lang="fr"` améliore l'accessibilité et le référencement.
- `<meta charset="utf-8">` garantit que les caractères accentués s'affichent correctement.
- `<meta name="viewport" …>` permet un affichage correct sur mobile.

Réf. Structure d'un document HTML - MDN.

II-3 Les balises de contenu texte

Les titres

HTML propose six niveaux de titres : `<h1>` à `<h6>`. Le `<h1>` est le titre principal de ta page et il est recommandé d'en avoir un seul. Les titres suivants (`<h2>`, `<h3>`, etc.) structurent les sous-parties de manière hiérarchique. Important : les titres servent à **structurer** le document, pas à définir la taille du texte (c'est le rôle du CSS).

Exemple :

```
<h1>Guide de randonnée</h1>
<h2>Équipement de base</h2>
<h3>Chaussures</h3>
<h3>Sac à dos</h3>
<h2>Itinéraires</h2>
```

Les paragraphes

Un paragraphe de texte s'écrit avec `<p>`. Chaque paragraphe est indépendant :

```
<p>
  Le parc national offre des paysages variés, accessibles à tous les
niveaux.
</p>
```

Il est interdit d'imbriquer des `<p>` l'un dans l'autre.

Les retours à la ligne et séparations

La balise `<br>` force un retour à la ligne **dans un même paragraphe** (utile pour des adresses ou de la poésie). Exemple :

```
<p>12 rue des Fleurs<br />80000 Amiens</p>
```

Pour marquer une séparation thématique entre deux sections, on utilise `<hr>`. Il ne s'agit pas d'une « ligne décorative », mais d'un changement de sujet.

```
<p>Introduction au guide…</p>
<hr />
<p>Partie 1 : Préparer sa sortie.</p>
```

Voir balises de texte HTML - MDN.

II-4 Mettre en valeur du contenu

Le HTML offre plusieurs balises pour souligner l'importance d'un mot ou d'une phrase :

- `<strong>` indique une **importance forte** (rendu en gras par défaut).
- `<em>` marque une **emphase** (souvent italique). Ces deux balises peuvent s'imbriquer.
- `<mark>` surligne un passage pertinent (par exemple un mot clé dans un résultat de recherche).
- `<small>` sert pour des notes annexes ou des mentions légales.
- `<del>` et `<ins>` marquent respectivement du contenu supprimé et du contenu ajouté, pratique pour montrer des modifications.

Exemple combiné :

```
<p>N'oublie pas : <strong>prends de l'eau</strong> en quantité suffisante.
</p>
<p>Cet itinéraire est <em>particulièrement</em> exposé au vent.</p>
<p>Ancien prix : <del>20 €</del> Nouveau prix : <ins>15 €</ins></p>
```

Réf. Éléments HTML de mise en valeur - MDN.

II-5 Les listes

Il existe deux types principaux de listes :

- `<ul>` pour une liste à puces (ordre non important).
- `<ol>` pour une liste numérotée (ordre important).

Chaque élément est un `<li>`.

Exemple de liste à puces :

```
<ul>
  <li>Eau</li>
  <li>Carte</li>
  <li>Lampe frontale</li>
</ul>
```

Exemple de liste ordonnée :

```
<ol start="3">
  <li>Échauffement</li>
  <li>Montée</li>
  <li>Sommet</li>
</ol>
```

On peut aussi imbriquer des listes dans un `<li>` pour créer des sous-listes, mais il vaut mieux éviter de multiplier les niveaux pour la lisibilité.

Réf. Listes HTML - MDN.

II-6 Les liens hypertextes

La balise `<a>` permet de créer un lien. L'attribut principal est `href`, qui indique la destination.

- Un lien absolu : `<a href="https://www.example.org/">Site officiel</a>`
- Un lien relatif : `<a href="../images/carte.pdf">Carte du site</a>`
- Un lien vers une ancre interne : `<a href="#top">Retour en haut</a>`

On peut aussi utiliser des schémas comme `mailto:` ou `tel:`.

Pour ouvrir un lien dans un nouvel onglet, on ajoute `target="_blank"`. Mais il faut aussi penser à la sécurité : associe toujours `rel="noopener noreferrer"`. nooopener noreferrer

```
<a href="https://www.example.org" target="_blank" rel="noopener
noreferrer">
  Ouvrir dans un nouvel onglet
</a>
```

Réf. Balise `<a>` - MDN.

II-7 Les images

La balise `<img>` affiche une image. Elle a deux attributs essentiels :

- `src` : l'URL du fichier image.
- `alt` : un texte alternatif pour décrire l'image aux personnes qui ne peuvent pas la voir.

Exemple :

```
<img src="images/sommet.jpg" alt="Vue panoramique depuis le sommet enneigé"
/>
```

Si l'image est purement décorative, l'attribut `alt` doit être vide : `alt=""`.

Pour améliorer les performances et l'accessibilité :

- Indique `width` et `height` pour éviter les sauts de mise en page.
- Utilise `loading="lazy"` pour différer le chargement des images en dehors de l'écran.
- Fournis plusieurs tailles avec `srcset` pour que le navigateur choisisse la meilleure version selon l'appareil.

Exemple d'image optimisée :

```
<img
  src="images/sommet-1200.jpg"
  srcset="
    images/sommet-600.jpg   600w,
    images/sommet-1200.jpg 1200w,
    images/sommet-2000.jpg 2000w
  "
  sizes="(max-width: 800px) 100vw, 800px"
  alt="Vue panoramique depuis le sommet"
  width="1200"
  height="800"
  loading="lazy"
  decoding="async"
/>
```

Pour ajouter une légende, on utilise `<figure>` et `<figcaption>` :

```
<figure>
  <img
    src="images/refuge.jpg"
    alt="Refuge en pierre au bord du lac"
    width="1200"
    height="800"
  />
  <figcaption>Refuge des Aiguilles, altitude 2150 m.</figcaption>
</figure>
```

Réf. Images en HTML - MDN.

II-8 Comment éviter les pièges fréquents

- N'utiliser `<h1>` qu'une seule fois, et respecter la hiérarchie.
- Ne pas imbriquer de `<p>` dans un autre `<p>`.
- N'utiliser `<br>` que pour des retours à la ligne sémantiques, jamais pour « faire de la place ».
- Ne pas transformer `<hr>` en élément décoratif.
- Rédiger des liens avec un texte explicite (évite « cliquez ici »).
- Ne jamais oublier `rel="noopener noreferrer"` avec `target="_blank"`.
- Fournir un `alt` pertinent pour chaque image informative.

II-9 Fiche-mémo

- **Structure** : doctype, langue, encodage, viewport, titre.
- **Texte** : `<p>`, `<h1>`…`<h6>`, `<br>`, `<hr>`.
- **Mise en valeur** : `<strong>`, `<em>`, `<mark>`, `<small>`, `<del>`, `<ins>`.
- **Listes** : `<ul>` (puces), `<ol>` (numéros), `<li>` (éléments).
- **Liens** : `<a href="…">`, penser à `rel="noopener noreferrer"` avec `_blank`.
- **Images** : toujours `alt`, spécifier `width`/`height`, utiliser `loading="lazy"`.

III Structure sémantique et organisation

III-1 Objectifs pédagogiques

Dans cette séance, nous apprendrons à organiser une page web de manière claire et cohérente grâce aux **balises sémantiques HTML5**. Nous verrons dans quels cas utiliser un simple `<div>` et dans quels cas il est préférable d'employer des balises comme `<header>`, `<main>`, `<article>`, `<section>` ou `<footer>`. Nous comprendrons aussi pourquoi la sémantique est essentielle pour l'**accessibilité** (repères ARIA implicites) et pour le **référencement naturel (SEO)**. Enfin, nous aborderons la bonne utilisation des titres hiérarchiques et l'imbrication correcte des sections.

III-2 `<div>` ou balises sémantiques : comment choisir ?

Le rôle de `<div>`

La balise `<div>` est un conteneur **générique**. Elle ne donne aucune indication sur le rôle de son contenu. Elle est utile pour regrouper des éléments en vue de leur appliquer un style CSS ou un comportement JavaScript. Cependant, si une balise plus spécifique existe, il est préférable de l'utiliser.

Exemple :

```
```

```
<div class="carte-produit">
  <h3>Stagiaire HTML – Promo 2025</h3>
  <p>Description…</p>
</div>
```

Les balises sémantiques HTML5

Depuis HTML5, un ensemble de balises permet de mieux décrire le rôle du contenu :

- `<header>` : en-tête d'une page ou d'une section. Peut contenir un logo, un titre, une accroche, un menu lié.
- `<footer>` : informations de fin de section ou de page (auteur, mentions légales, ©).
- `<main>` : contenu principal de la page (unique et non imbriqué).
- `<nav>` : zone de navigation regroupant des liens importants (menu, pagination).
- `<article>` : contenu autonome et réutilisable (article, fiche produit, commentaire).
- `<section>` : regroupement thématique avec un titre. Si aucun titre pertinent n'existe, un `<div>` suffit.
- `<aside>` : contenu complémentaire, connexe au contenu principal (encadré, glossaire, pub, bio d'auteur).

Réf. Balises sémantiques HTML - MDN.

Règle d'or : si une balise exprime clairement le rôle du contenu, utilise-la plutôt qu'un `<div>`.

III-3 Accessibilité et SEO : pourquoi la sémantique est cruciale

Accessibilité et repères ARIA implicites

Les balises sémantiques fournissent automatiquement des **landmarks** (repères) aux technologies d'assistance. Cela permet à un lecteur d'écran de naviguer efficacement dans une page. Par exemple :

- `<main>` est reconnu comme rôle **main**.
- `<nav>` comme rôle **navigation**.
- `<aside>` comme rôle **complementary**.
- `<footer>` comme rôle **contentinfo**.
- `<article>` comme rôle **article**.
- `<section>` comme rôle **region**, uniquement si elle a un titre ou un attribut accessible (`aria-label`).

Ainsi, un utilisateur de lecteur d'écran peut « sauter » directement à une section comme nous le faisons visuellement.

Bonnes pratiques d'accessibilité

- Un seul `<main>` par page.
- Chaque `<section>` doit avoir un titre visible ou un nom accessible :

```
<section aria-labelledby="faq-title">
  <h2 id="faq-title">FAQ</h2>
  …
</section>
```

- Ajouter un **lien d'évitement** tout en haut de la page pour aller directement au contenu principal :

```
<a href="#contenu" class="skip">Aller au contenu</a>
<main id="contenu">…</main>
```

- Rédiger des liens explicites : « Voir les tarifs » est plus clair que « Cliquez ici ».
- Éviter de doubler les rôles ARIA : un `<nav>` n'a pas besoin de `role="navigation"`.

Réf. HTML et accessibilité - MDN.

SEO et structure

Une page bien structurée aide les moteurs de recherche à comprendre le contenu. L'usage correct des balises sémantiques, combiné à une hiérarchie claire des titres, facilite l'indexation. Attention cependant : le SEO dépend avant tout de la qualité du contenu, des liens entrants et de la performance technique. La sémantique est un atout, mais pas une solution magique.

III-4 Imbrication et hiérarchie

Principes d'imbrication

- On peut avoir plusieurs `<header>` ou `<footer>` si chacun se rapporte à une section distincte.
- On ne doit avoir **qu'un seul `<main>`** par page.
- `<nav>` peut apparaître au niveau global (menu principal) ou local (pagination).
- Un `<article>` peut contenir son propre `<header>`, `<footer>` et des `<section>`.

Hiérarchie des titres

Un document doit respecter une structure hiérarchique :

- La page commence avec un `<h1>`.
- Les grandes sections utilisent `<h2>`.

- Les sous-sections utilisent `<h3>`, et ainsi de suite.

Bien que techniquement plusieurs `<h1>` soient possibles, il est plus simple et plus clair d'avoir un seul `<h1>` global et de descendre ensuite dans les niveaux.

III-5 Exemples de structures

Squelette complet d'une page

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <title>Titre de la page</title>
    <meta name="viewport" content="width=device-width, initial-scale=1" />
  </head>
  <body>
    <a href="#main" class="skip">Aller au contenu</a>

    <header>
      <h1>Nom du site</h1>
      <nav aria-label="Navigation principale">
        <ul>
          <li><a href="/">Accueil</a></li>
          <li><a href="/articles">Articles</a></li>
          <li><a href="/contact">Contact</a></li>
        </ul>
      </nav>
    </header>

    <main id="main">
      <article>
        <header>
          <h2>Titre de l'article</h2>
          <p>Publié le <time datetime="2025-09-06">6 septembre 2025</time>
</p>
        </header>
        <section>
          <h3>Contexte</h3>
          <p>Texte d'introduction…</p>
        </section>
        <footer>
          <p>Auteur : <a href="/auteurs/manuel">Manuel</a></p>
        </footer>
      </article>
      <aside>
        <h2>À retenir</h2>
        <p>Résumé, glossaire, lien connexe…</p>
      </aside>
    </main>

    <footer>
```

```
        <p>© 2025 - Mon site. <a href="/mentions-legales">Mentions
légales</a></p>
    </footer>
  </body>
</html>
```

Réf. Guide des balises sémantiques - MDN.

III-6 Pièges fréquents

- Utiliser `<section>` sans titre : dans ce cas, préfère `<div>`.
- Multiplier les `<nav>` pour des simples liens isolés.
- Créer plusieurs `<main>` ou l'imbriquer dans d'autres éléments.
- Ajouter inutilement des rôles ARIA déjà implicites.
- Employer `<header>` ou `<footer>` juste pour appliquer du style.
- Sauter des niveaux de titres (ex. passer de `<h1>` à `<h4>` sans `<h2>` ni `<h3>`).
- Tomber dans le « divitis » en utilisant uniquement `<div>` partout.

III-7 Fiche-mémo

- Utilise les balises sémantiques quand c'est possible : `<header>`, `<main>`, `<nav>`, `<article>`, `<section>`, `<aside>`, `<footer>`.
- `<div>` est réservé aux cas où aucune balise sémantique n'est pertinente.
- `<main>` doit être unique et non imbriqué.
- `<section>` doit avoir un titre ; sinon, utilise `<div>`.
- `<article>` représente un contenu autonome.
- Les balises sémantiques exposent des repères implicites pour l'accessibilité.
- Une bonne hiérarchie de titres (`h1 > h2 > h3 …`) rend le contenu clair pour les humains comme pour les moteurs de recherche.

IV Métadonnées et ressources externes

IV-1 Objectifs pédagogiques

Dans cette séance, nous apprendrons à configurer correctement l'élément `<head>` d'une page HTML. Nous verrons quelles sont les **métadonnées essentielles** (encodage, affichage mobile, titre, description), comment relier des ressources externes (feuilles de style, favicon), et enfin comment charger du JavaScript avec de bonnes pratiques modernes (`defer`, `async`, `type="module"`).

IV-2 L'élément `<head>` : rôle et ordre recommandé

Le `<head>` regroupe toutes les **informations qui décrivent la page** mais qui ne sont pas directement affichées comme contenu principal. On y trouve le titre d'onglet, les feuilles de style, les favicons, des scripts et les métadonnées utiles aux navigateurs et aux moteurs de recherche.

Un ordre recommandé pour que tout fonctionne bien est le suivant :

1. Déclaration du `<!DOCTYPE html>` et de `<html lang="fr">`.
2. Ouverture du `<head>`.
3. Les métadonnées essentielles :
   - `<meta charset="utf-8">`
   - `<meta name="viewport" …>`
   - `<title>` et `<meta name="description" …>`
4. Les icônes et manifest si nécessaire.
5. Les feuilles CSS.
6. Les scripts critiques si besoin immédiat, ou mieux : des scripts avec `defer` ou placés à la fin du `<body>`.

Réf. L'élément `<head>` - MDN.

IV-3 Les métadonnées essentielles

Encodage (`charset`)

La première chose à mettre dans `<head>` est l'encodage. L'instruction :

```
<meta charset="utf-8" />
```

assure que tous les caractères, y compris les accents, seront affichés correctement. Réf. Définir l'encodage - MDN.

Affichage mobile (`viewport`)

Sur mobile, si on ne fait rien, les navigateurs appliquent un zoom par défaut. Pour contrôler l'affichage, on utilise :

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

Cela fixe la largeur logique à la largeur de l'écran et un zoom initial à 100 %. Évite de bloquer le zoom utilisateur (`user-scalable=no`), car c'est un problème d'accessibilité.

Réf. Meta viewport - MDN.

Titre (`<title>`)

Chaque page doit avoir un titre unique et descriptif, d'environ 50 à 60 caractères. Ce titre apparaît dans l'onglet du navigateur et sert aussi aux moteurs de recherche.

```
<title>Initiation à HTML5 – Cours 1</title>
```

Description SEO

Une balise optionnelle mais très utile est la **description** :

```
<meta
  name="description"
  content="Découvre les bases de HTML5 : structure, balises de texte et
bonnes pratiques."
/>
```

Elle n'améliore pas directement le classement SEO, mais elle influence le **taux de clic** car elle peut être affichée comme extrait dans les résultats de recherche.

IV-4 Relier des ressources avec `<link>`

Feuille de style CSS

Pour appliquer des styles :

```
<link rel="stylesheet" href="/assets/css/style.css" />
```

On peut préciser un attribut `media` si la feuille n'est utile que dans certains cas (par ex. impression).

Réf. Élément `<link>` - MDN.

Favicons et icônes

Les favicons sont les petites icônes visibles dans l'onglet. On peut fournir un **SVG** moderne et un **PNG** de secours :

```
<link
  rel="icon"
  href="/assets/icons/favicon.svg"
  type="image/svg+xml"
  sizes="any"
/>
<link
  rel="icon"
```

```
    href="/assets/icons/favicon-32.png"
    type="image/png"
    sizes="32x32"
  />
```

Il est aussi possible d'ajouter une **apple-touch-icon** pour iOS.

IV-5 Charger du JavaScript avec `<script>`

Cas de base

La forme la plus simple est :

```
<script src="/assets/js/app.js"></script>
```

Mais attention : sans attribut, le chargement est bloquant. C'est pourquoi on préfère des alternatives.

Les bonnes pratiques

- `defer` : télécharge le script en parallèle et l'exécute à la fin du parsing HTML, dans l'ordre des balises. C'est le choix recommandé pour la plupart des cas.

```
<script src="/assets/js/app.js" defer></script>
```

- `async` : le script est exécuté dès qu'il est prêt, sans garantir l'ordre. À utiliser pour des scripts indépendants comme de l'analytics.

```
<script src="/assets/js/analytics.js" async></script>
```

- `type="module"` : permet d'écrire en JavaScript moderne avec `import` et `export`. Les modules se chargent en différé par défaut.

```
<script type="module" src="/assets/js/main.js"></script>
```

Réf. Élément `<script>` - MDN.

IV-6 Exemple de `<head>` bien structuré

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>

    <title>HTML5 - Métadonnées et ressources</title>
    <meta
      name="description"
      content="Configurer le head d'une page HTML5 : encodage, viewport,
titre, description SEO, CSS, favicon et scripts."
    />

    <!-- Favicon -->
    <link
      rel="icon"
      href="/assets/icons/favicon.svg"
      type="image/svg+xml"
      sizes="any"
    />
    <link
      rel="icon"
      href="/assets/icons/favicon-32.png"
      type="image/png"
      sizes="32x32"
    />

    <!-- Feuille de style -->
    <link rel="stylesheet" href="/assets/css/style.css" />

    <!-- Script différé -->
    <script src="/assets/js/app.js" defer></script>
  </head>
  <body>
    <h1>Page de démonstration</h1>
    <p>Exemple avec métadonnées, favicon, CSS et JS.</p>
  </body>
</html>
```

IV-7 Pièges fréquents

- Mettre `<meta charset>` trop tard dans `<head>`, ce qui cause des caractères illisibles.
- Bloquer le zoom utilisateur dans `viewport`.
- Réutiliser le même titre ou la même description partout.
- Oublier la favicon ou ne pas fournir de format de secours.
- Charger des scripts bloquants dans `<head>` sans `defer` ou `async`.

IV-8 Fiche-mémo

- `<head>` contient les métadonnées et ressources, pas le contenu affiché.
- Ordre recommandé : charset → viewport → title → description → icônes → CSS → scripts.

- Toujours déclarer `<meta charset="utf-8">` en premier.
- Le viewport standard : `width=device-width, initial-scale=1.0`.
- Chaque page doit avoir un `<title>` unique.
- Une description SEO claire améliore le snippet dans les résultats.
- Fournis au moins une favicon fonctionnelle.
- Préfère `defer` ou `type="module"` pour charger du JavaScript.

V Introduction au CSS

V-1 Objectifs pédagogiques

Dans cette séance, nous allons découvrir le **CSS** (*Cascading Style Sheets*), le langage qui permet de mettre en forme une page web. Nous comprendrons en quoi il se distingue du **HTML**, comment appliquer du style à une page par différentes méthodes, et comment cibler les bons éléments grâce aux **sélecteurs**. Nous apprendrons à utiliser les propriétés de base : couleur, arrière-plan, police, tailles, marges, espacements, bordures. Enfin, nous mettrons en pratique en stylisant une petite page complète.

V-2 HTML et CSS : deux rôles complémentaires

Le **HTML** donne la structure et le sens du contenu (titres, paragraphes, listes, images). Le **CSS**, lui, définit la présentation : couleurs, polices, tailles, alignement, mise en page.

Cette séparation des rôles est capitale : le HTML décrit « ce que dit le contenu », le CSS décrit « comment il apparaît ». Réf. Qu'est-ce que CSS ? - MDN.

V-3 Trois manières d'ajouter du style

1. Style en ligne (inline)

Nous pouvons ajouter du style directement dans une balise grâce à l'attribut `style`.

```
<h1 style="color: darkblue; text-align: center;">Titre</h1>
```

C'est rapide pour tester, mais à éviter en production : cela surcharge le HTML, rend le code difficile à maintenir et empêche la réutilisation.

2. Style interne

On peut insérer un bloc `<style>` dans le `<head>` du document :

```
<head>
  <style>
    h1 {
```

```
      color: darkblue;
      text-align: center;
    }
  </style>
</head>
```

C'est pratique pour un petit fichier isolé, mais cela ne sépare pas vraiment contenu et présentation à l'échelle d'un site entier.

3. Style externe

La bonne pratique consiste à mettre le CSS dans un fichier séparé (`.css`) relié via `<link>`.

```
<link rel="stylesheet" href="assets/css/style.css" />
```

```
/* assets/css/style.css */
h1 {
  color: darkblue;
  text-align: center;
}
```

Cela permet de réutiliser la même feuille de style sur plusieurs pages, et le navigateur peut la mettre en cache. Réf. Les différentes façons d'appliquer du CSS - MDN.

V-4 Cibler des éléments avec les sélecteurs

Par balise

```
p {
  line-height: 1.6;
}
h1 {
  margin-bottom: 0.5rem;
}
```

Cela s'applique à tous les éléments de ce type.

Par classe

Une classe peut être utilisée sur plusieurs éléments :

```
<p class="intro">Bienvenue…</p>
```

```
.intro {
  font-style: italic;
}
```

Par identifiant (id)

Un identifiant est unique dans la page :

```
<h1 id="titre-principal">Accueil</h1>
```

```
#titre-principal {
  text-transform: uppercase;
}
```

Autres combinaisons

- **Groupement** : `h1, h2, h3 { font-family: sans-serif; }`
- **Descendant** : `nav a { text-decoration: none; }`
- **Type + classe** : `p.notice { background: #fffae6; }`

Réf. Les sélecteurs CSS - MDN.

V-5 Propriétés CSS essentielles

Couleur (`color`)

```
p {
  color: #333;
}
h1 {
  color: rgb(20, 20, 120);
}
h2 {
  color: hsl(230, 60%, 40%);
}
```

Réf. color - MDN.

Arrière-plan (`background`)

```
body {
  background: #f5f7fa;
}
header {
  background: #0b3d91;
  color: white;
}
```

Réf. background - MDN.

Police (`font-family`)

Toujours prévoir une pile de secours :

```
body {
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, Arial, sans-
serif;
}
```

Taille (`font-size`)

Mieux vaut utiliser des unités relatives (`rem`, `em`) plutôt que des pixels pour faciliter l'accessibilité :

```
h1 {
  font-size: 2rem;
}
p {
  font-size: 1rem;
  line-height: 1.6;
}
```

Alignement (`text-align`)

```
h1 {
  text-align: center;
}
p {
  text-align: justify;
}
```

Marges et espacements (`margin`, `padding`)

```
p {
  margin: 1rem 0;
}
.section {
  padding: 1rem 1.5rem;
}
```

Réf. margin - MDN.

Bordures (`border`)

```
.box {
  border: 1px solid #ddd;
}
.box-important {
  border: 2px dashed #0b3d91;
}
```

Réf. border - MDN.

V-6 Exemple complet

index1.html

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <title>Demo CSS – Bases</title>
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link rel="stylesheet" href="assets/css/style.css" />
  </head>
  <body>
    <h1 class="titre">Bienvenue</h1>
    <p class="intro">Ceci est un paragraphe d’introduction.</p>
    <p>Un autre paragraphe, pour tester marges et couleurs.</p>
  </body>
</html>
```

assets/css/style.css

```
html {
  font-size: 100%;
}
```

```
body {
  margin: 0;
  padding: 0 1rem;
  font-family: system-ui, sans-serif;
  color: #222;
  background: #f7f9fc;
}

.titre {
  text-align: center;
  font-size: 2rem;
  margin: 1.5rem 0 1rem;
  color: #0b3d91;
}

p {
  font-size: 1rem;
  line-height: 1.6;
  margin: 1rem 0;
}

.intro {
  font-style: italic;
  background: #fffae6;
  padding: 0.5rem 0.75rem;
  border-left: 4px solid #f5c518;
}
```

V-7 Fiche-mémo

- **HTML structure le contenu**, CSS définit la présentation.
- Méthodes d'intégration : inline (éviter), interne (limité), externe (recommandé).
- Sélecteurs de base : balise, `.classe`, `#id`.
- Spécificité : inline > id > classe > balise.
- Propriétés essentielles : `color`, `background`, `font-family`, `font-size`, `line-height`, `text-align`, `margin`, `padding`, `border`.
- Utilise `rem` pour les tailles, pense à `line-height` pour la lisibilité.

VI Mise en page moderne (HTML + CSS)

VI-1 Objectifs pédagogiques

À la fin de cette séance, nous saurons :

- expliquer et manipuler le **modèle de boîte** (content, padding, border, margin) ;
- distinguer les comportements **bloc**, **inline** et **inline-block** ;

- construire des mises en page avec **Flexbox** (axes, alignements, `gap`, propriétés `flex`) ;
- comprendre les bases de **CSS Grid** pour créer des grilles réactives ;
- réaliser une page avec **en-tête**, **zone de contenu en deux colonnes** et **pied de page**, en rendant le tout responsive.

VI-2 Le modèle de boîte (Box Model)

En CSS, chaque élément est représenté comme une boîte composée de quatre zones :

```
┌──────────────────────────────────────────────┐
│                   margin                     │
│   ┌──────────────────────────────────────┐   │
│   │                 border               │   │
│   │   ┌──────────────────────────────┐   │   │
│   │   │            padding           │   │   │
│   │   │   ┌──────────────────────┐   │   │   │
│   │   │   │        content       │   │   │   │
│   │   │   └──────────────────────┘   │   │   │
│   │   └──────────────────────────────┘   │   │
│   └──────────────────────────────────────┘   │
└──────────────────────────────────────────────┘
```

1. le **contenu** (`content`) : texte, image ou autre média,
2. le **padding** : espace **interne** entre le contenu et la bordure,
3. la **bordure** (`border`),
4. la **marge** (`margin`), qui est l'espace **externe** séparant les boîtes entre elles.

Réf. Box model - MDN.

Largeur et hauteur calculées

Par défaut (`box-sizing: content-box`), la largeur totale d'une boîte est calculée comme :

```
largeur totale = width + padding + border
```

Les marges s'ajoutent à l'extérieur.

Pour simplifier les calculs, la bonne pratique moderne consiste à utiliser :

```
*,
*::before,
*::after {
```

```
    box-sizing: border-box;
  }
```

Avec `border-box`, la largeur et la hauteur incluent déjà le padding et la bordure.

Cas particuliers des marges

Les marges verticales peuvent se **fusionner** (margin collapsing). Cela signifie que si deux blocs verticaux se suivent, la marge finale correspond à la plus grande des deux, et non à leur somme.

Ce phénomène se produit entre deux frères, ou entre un parent sans padding/border et son premier ou dernier enfant. Mais il ne s'applique pas aux éléments flex ou grid.

Autre point à retenir : `margin: auto` permet de centrer un bloc si sa largeur est définie.

VI-3 Bloc, inline et inline-block

`display: block`

Un élément de type bloc occupe toute la largeur disponible et commence sur une nouvelle ligne. Il accepte `width`, `height`, `margin` et `padding`. Les titres (`h1` à `h6`), paragraphes, sections, listes et divs sont des éléments bloc.

`display: inline`

Un élément inline s'insère dans le flux du texte, comme un mot. Il ne respecte pas `width` et `height`, et les marges verticales sont peu ou pas prises en compte. Les balises `<span>`, `<a>`, `<em>`, `<strong>` ou encore `<img>` (par défaut) sont inline.

`display: inline-block`

Un élément inline-block se comporte comme un mot dans la ligne, mais il accepte `width`, `height`, `margin` et `padding`. Il est utile pour disposer des petites boîtes côte à côte. Attention cependant : les espaces dans le HTML entre deux inline-block produisent un petit espacement visible, ce qui peut nécessiter des ajustements. Aujourd'hui, Flexbox et Grid offrent des solutions plus simples.

Réf. display - MDN.

VI-4 Introduction à Flexbox

Concepts de base

Avec `display: flex`, un élément devient un conteneur flex. Ses enfants deviennent des **flex items** disposés le long d'un axe principal (`main axis`) et d'un axe transversal (`cross axis`). Par défaut, l'axe principal est horizontal (de gauche à droite).

Propriétés du conteneur

Un conteneur flex accepte des propriétés comme :

```
.container {
  display: flex;
  flex-direction: row; /* orientation */
  flex-wrap: wrap; /* retour à la ligne éventuel */
  gap: 1rem; /* espace entre les items */
  justify-content: center; /* alignement sur l'axe principal */
  align-items: stretch; /* alignement sur l'axe transversal */
}
```

- `justify-content` gère l'alignement horizontal (flex-start, center, space-between, etc.).
- `align-items` gère l'alignement vertical des items.
- `gap` crée un espace intégré entre items (préférable aux marges).

Propriétés des items

Chaque enfant d'un flex peut définir sa flexibilité avec :

```
.item {
  flex: 1 1 auto;
}
```

Le raccourci `flex` combine trois valeurs :

- `flex-grow` : capacité à grandir si de l'espace reste,
- `flex-shrink` : capacité à rétrécir si l'espace manque,
- `flex-basis` : taille de base avant ajustement.

Exemple :

```
.col-contenu {
  flex: 1 1 0;
} /* colonne fluide */
.col-aside {
  flex: 0 0 320px;
} /* colonne fixe */
```

On peut aussi changer l'ordre d'affichage avec `order` ou personnaliser l'alignement d'un item avec `align-self`.

Réf. Flexbox - MDN.

VI-5 Premiers pas avec CSS Grid

Les bases

Avec `display: grid`, nous créerons une grille 2D composée de lignes et de colonnes. Les enfants se placent automatiquement ou peuvent être positionnés précisément.

Exemple simple à deux colonnes :

```
.grid {
  display: grid;
  grid-template-columns: 2fr 1fr;
  gap: 1rem;
}
```

Ici, la première colonne occupe deux fois plus d'espace que la seconde.

Unités et fonctions utiles

- `fr` représente une fraction de l'espace disponible.
- `minmax(min, max)` définit une borne adaptable.
- `auto-fit` et `auto-fill` permettent de créer des grilles réactives qui s'ajustent automatiquement.

Exemple de cartes réactives :

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
```

Réf. CSS Grid - MDN.

VI-6 Exemple de mise en page complète

Voici un gabarit basé sur Flexbox avec en-tête, contenu en deux colonnes et pied de page.

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8" />
    <title>Layout 2 colonnes - Flexbox</title>
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link rel="stylesheet" href="assets/css/style.css" />
  </head>
  <body>
    <header class="header">
      <div class="container">
```

```
        <h1>Mon site</h1>
        <p>Un sous-titre concis</p>
      </div>
    </header>

    <main class="container">
      <div class="main">
        <article class="col-contenu">
          <h2>Article principal</h2>
          <p>Texte de démonstration. Cette colonne est fluide.</p>
        </article>

        <aside class="col-aside">
          <h2>À côté</h2>
          <p>Colonne latérale fixe (~320px).</p>
        </aside>
      </div>
    </main>

    <footer class="footer">
      <div class="container">
        <p>© 2025 - Mon site</p>
      </div>
    </footer>
  </body>
</html>
```

Et la feuille CSS correspondante :

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #222;
  background: #f7f9fc;
}

.container {
  max-width: 1000px;
  margin: 0 auto;
  padding: 1rem;
}

.header,
.footer {
  background: #0b3d91;
  color: white;
```

```
    padding: 1rem;
  }

  .main {
    display: flex;
    gap: 1rem;
    padding: 1rem 0;
  }

  .col-contenu {
    flex: 1 1 0;
    background: white;
    padding: 1rem;
  }

  .col-aside {
    flex: 0 0 320px;
    background: white;
    padding: 1rem;
  }

  @media (max-width: 800px) {
    .main {
      flex-direction: column;
    }
    .col-aside {
      flex-basis: auto;
    }
  }
```

On peut obtenir la même structure avec Grid grâce à `grid-template-areas`.

Réf.Media Queries

VI-7 Pièges fréquents

- Oublier `box-sizing: border-box`, ce qui complique les calculs de largeur.
- Utiliser des marges pour espacer les flex items au lieu de `gap`.
- Penser que les marges verticales s'additionnent toujours : elles peuvent fusionner.
- Attribuer des largeurs fixes trop rigides au lieu d'utiliser `%`, `fr` ou `minmax`.
- Mélanger des réglages contradictoires (`order`, `width`, `flex-basis`).

VI-8 Fiche-mémo

- **Box model** : content → padding → border → margin.
- Toujours appliquer `box-sizing: border-box`.
- **Display** :

- block : occupe toute la ligne,
- inline : s'insère dans le texte,
- inline-block : hybride.
- **Flexbox** : simple et puissant pour les mises en page en ligne ou en colonne.
- **Grid** : idéal pour les grilles 2D complexes et réactives.
- **Responsive** : privilégier unités flexibles et media queries.

VII Médias et formulaires

VII-1 Objectifs pédagogiques

Cette séance a deux grands axes. D'une part, nous allons apprendre à intégrer des **médias** dans une page web avec `<audio>`, `<video>` et `<iframe>`, tout en respectant les bonnes pratiques d'accessibilité, de performance et de sécurité. D'autre part, nous découvrirons comment construire des **formulaires HTML** modernes et accessibles : balises de base, différents types de champs HTML5, attributs de validation, et création d'un petit formulaire de contact complet.

VII-2 Insertion de médias

L'audio avec `<audio>`

La balise `<audio>` permet d'intégrer du son directement dans une page. Nous pouvons fournir plusieurs formats pour maximiser la compatibilité (MP3, Ogg, WAV). L'attribut `controls` est indispensable pour afficher l'interface native du navigateur. Il est possible aussi définir un comportement particulier avec `loop`, `muted`, ou `preload="metadata"`.

Exemple :

```
<audio controls preload="metadata">
  <source src="media/jingle.mp3" type="audio/mpeg" />
  <source src="media/jingle.ogg" type="audio/ogg" />
  Votre navigateur ne supporte pas l'audio HTML5.
</audio>
```

Réf. Balise `<audio>` - MDN.

La vidéo avec `<video>`

Le principe est similaire. La balise `<video>` accepte des formats comme MP4/H.264, WebM ou Ogg/Theora. Il est recommandé de préciser `width` et `height` pour éviter les sauts de mise en page, et d'ajouter un visuel d'attente avec `poster`.

Un point important est l'accessibilité : pense à fournir des sous-titres via `<track>` au format WebVTT.

Exemple :

```
<video
  controls
  width="800"
  height="450"
  preload="metadata"
  poster="media/apercu.jpg"
>
  <source src="media/demo.webm" type="video/webm" />
  <source src="media/demo.mp4" type="video/mp4" />
  <track
    kind="subtitles"
    srclang="fr"
    label="Français"
    src="media/demo-fr.vtt"
    default
  />
  Votre navigateur ne supporte pas la vidéo HTML5.
</video>
```

Réf. Balise `<video>` - MDN.

Les iframes `<iframe>`

L'iframe sert à intégrer une ressource externe (une carte, une vidéo YouTube, un widget). Mais attention : elle peut introduire des failles de sécurité et poser des problèmes d'accessibilité.

Pour limiter les risques, ajoute toujours un `title`, pense à `loading="lazy"` pour les performances, et utilise l'attribut `sandbox` pour restreindre ce que peut faire le contenu embarqué.

Exemple :

```
<div class="embed">
  <iframe
    src="https://www.example.com/carte"
    title="Carte interactive de la zone"
    loading="lazy"
    sandbox="allow-scripts allow-same-origin"
    referrerpolicy="strict-origin-when-cross-origin"
  >
  </iframe>
</div>

<style>
  .embed {
    max-width: 900px;
    margin: 1rem auto;
  }
  .embed iframe {
```

```
      width: 100%;
      aspect-ratio: 16 / 9;
      border: 0;
    }
  </style>
```

Réf. Balise `<iframe>` - MDN.

VII-3 Formulaires HTML : les bases

Un formulaire commence avec `<form>`. C'est cette balise qui regroupe les champs et qui définit l'URL de destination (`action`) ainsi que la méthode HTTP (`method="get"` ou `"post"`). Pour un envoi de fichiers, il faut ajouter `enctype="multipart/form-data"`.

Exemple minimal :

```
<form action="/contact" method="post">
  <div>
    <label for="fnom">Nom</label>
    <input type="text" id="fnom" name="nom" required />
  </div>
</form>
```

Chaque champ doit avoir un `name` (clé utilisée lors de l'envoi) et idéalement un `id` associé à un `<label>`. Le label rend le formulaire beaucoup plus accessible : cliquer dessus déplace le curseur dans le champ. Pour regrouper logiquement plusieurs champs, on peut utiliser `<fieldset>` et `<legend>`.

Réf. Les formulaires HTML - MDN.

VII-4 Les champs `<input>` et leurs variantes

Champs texte et dérivés

Les plus courants :

```
<input type="text" name="nom" required />
<input type="email" name="email" required />
<input type="url" name="site" />
<input type="tel" name="tel" inputmode="tel" />
```

Le type `email` ou `url` déclenche une validation de format automatique. Le type `tel` n'impose pas de validation mais propose un clavier adapté sur mobile.

Nombres et curseurs

```
<input type="number" name="age" min="0" max="120" step="1" />
<input type="range" name="note" min="0" max="10" step="1" value="5" />
```

Le premier affiche un champ numérique avec flèches, le second un slider.

Dates, heures et couleurs

HTML5 fournit plusieurs types pratiques :

```
<input type="date" name="rdv" />
<input type="time" name="heure" />
<input type="datetime-local" name="rdvcomplet" />
<input type="month" name="mois" />
<input type="week" name="semaine" />
<input type="color" name="couleur" value="#204ECF" />
```

Choix multiples : checkbox et radio

Pour des choix binaires ou exclusifs :

```
<input type="checkbox" id="cgu" name="cgu" required />
<label for="cgu">J'accepte les conditions</label>

<label><input type="radio" name="genre" value="f" /> Femme</label>
<label><input type="radio" name="genre" value="h" /> Homme</label>
```

Upload de fichiers

```
<input type="file" name="cv" accept=".pdf,.doc,.docx" />
```

À combiner avec `enctype="multipart/form-data"` sur le formulaire.

Réf. Types d'input HTML - MDN.

VII-5 Autres contrôles

Zone de texte

```
<label for="msg">Message</label>
<textarea id="msg" name="message" rows="6"></textarea>
```

Listes déroulantes

```
<select name="sujet" required>
  <option value="">- Choisir -</option>
  <option value="infos">Demande d'informations</option>
  <option value="support">Support technique</option>
</select>
```

Boutons

```
<button type="submit">Envoyer</button>
<button type="reset">Réinitialiser</button>
```

Attention : par défaut, un `<button>` dans un `<form>` est de type `submit`.

VII-6 Validation et accessibilité

Le HTML5 intègre une validation native. Des attributs comme `required`, `min`, `max`, `pattern` permettent de contrôler la saisie sans JavaScript. Exemple avec feedback visuel via CSS :

```
input:required:invalid {
  border-color: #d33;
}
input:required:valid {
  border-color: #2a7;
}
```

Pour l'accessibilité, il faut :

- toujours utiliser des `<label>` explicites ;
- prévoir des textes d'aide reliés par `aria-describedby` si nécessaire ;
- grouper logiquement les champs avec `<fieldset>` ;
- soigner le contraste et la visibilité du focus.

Réf. Validation des formulaires - MDN.

VII-7 Exemple de formulaire complet

Voici un formulaire de contact correct :

```
<form action="/contact" method="post">
  <fieldset>
    <legend>Contact</legend>
```

```
    <div>
      <label for="nom">Nom</label>
      <input type="text" id="nom" name="nom" required />
    </div>

    <div>
      <label for="email">E-mail</label>
      <input type="email" id="email" name="email" required />
    </div>

    <div>
      <label for="sujet">Sujet</label>
      <select id="sujet" name="sujet" required>
        <option value="">- Choisir -</option>
        <option value="infos">Demande d'informations</option>
        <option value="support">Support technique</option>
      </select>
    </div>

    <div>
      <label for="message">Message</label>
      <textarea
        id="message"
        name="message"
        rows="6"
        required
        minlength="10"
        maxlength="2000"
      ></textarea>
    </div>

    <div>
      <input type="checkbox" id="cgu" name="cgu" required />
      <label for="cgu">J'accepte les conditions</label>
    </div>

    <button type="submit">Envoyer</button>
  </fieldset>
</form>
```

VII-8 Fiche-mémo

- **Audio/vidéo** : toujours fournir `controls`, penser aux sous-titres (`<track>`), préciser `width`/`height` et utiliser `preload="metadata"`.
- **Iframe** : ajouter `title`, `loading="lazy"`, `sandbox` et éventuellement `referrerpolicy`.
- **Formulaires** : `action`, `method`, `enctype`, `autocomplete`.
- **Champs** : `name` obligatoire, `label` relié par `for/id`, `required` pour validation simple.
- **Types HTML5** : `email`, `url`, `tel`, `number`, `range`, `date/time`, `color`, `file`, `checkbox`, `radio`.

- **Accessibilité** : labels visibles, champs groupés avec `<fieldset>`, contrastes suffisants, focus bien visibles.
- Toujours valider **côté serveur**, même si HTML5 fait déjà une première vérification.

VIII Bonnes pratiques et outils

VIII-1 Objectifs pédagogiques

Dans cette séance, nous apprendrons à valider nottre code HTML et CSS pour éviter les erreurs courantes, à respecter les règles essentielles d'**accessibilité**, et à adopter les bonnes pratiques de structure et de lisibilité. Nous découvrirons également comment utiliser les **outils développeur** intégrés aux navigateurs modernes et où trouver de la documentation fiable pour progresser en autonomie.

VIII-2 Valider son code

La validation n'est pas une contrainte inutile : elle permet de détecter des erreurs de syntaxe, d'améliorer la compatibilité entre navigateurs, et d'assurer une base solide pour l'accessibilité et le référencement.

Le **W3C Validator** (validator.w3.org) reste l'outil de référence. Nous pouvons lui donner une URL, envoyer un fichier ou simplement coller ton code. Pour HTML5, il existe aussi **Nu Html Checker**, et pour le CSS, un validateur dédié (jigsaw.w3.org/css-validator).

Quelques erreurs typiques détectées par ces outils :

- balises non fermées (`<p>Texte`),
- attributs obsolètes (`<font color="red">`),
- mauvais encodage (`<meta charset="latin1">` au lieu de `utf-8`).

Réf. Validation HTML - MDN.

VIII-3 Accessibilité (A11y)

Un site accessible bénéficie à tous les utilisateurs, y compris les personnes qui utilisent un lecteur d'écran, qui naviguent au clavier ou qui ont des contraintes visuelles. En France, la loi impose le respect du **RGAA**, et au niveau européen, des directives d'accessibilité numérique s'appliquent.

- **Texte alternatif (`alt`)** : obligatoire pour toutes les images. Un logo aura comme alt le nom de l'entreprise ; une image décorative doit avoir `alt=""`.
- **Labels de formulaire** : chaque champ doit être relié à un `<label>`. Cliquer sur le label doit donner le focus au champ, ce qui est essentiel pour l'ergonomie et l'accessibilité.
- **Attributs ARIA** : ils complètent la sémantique HTML pour les composants interactifs complexes. Exemple : un bouton qui ouvre un menu peut indiquer son état avec `aria-expanded="false"` ou `true`.
- **Autres règles essentielles** : titres hiérarchisés (`h1`, puis `h2`…), contraste suffisant (au moins 4.5:1), navigation clavier fluide, éviter les textes en image.

Réf. Accessibilité HTML - MDN.

VIII-4 Bonnes pratiques de code

Un bon code se distingue autant par sa structure que par sa clarté.

Éviter le code obsolète

N'utilise plus `<font>`, `<center>`, `<b>` ou `<i>` pour la présentation. À la place, mise sur des balises sémantiques comme `<strong>` et `<em>` associées à du CSS.

Séparer contenu, style et comportement

- HTML = contenu et structure,
- CSS = présentation,
- JavaScript = interaction et logique.

Évite d'écrire du style inline (`style="…"`) ou du JavaScript directement dans les balises (`onclick="…"`) : cela rend le code confus et difficile à maintenir.

Lisibilité

Indente ton code (2 ou 4 espaces mais reste cohérent), utilise des noms de classes explicites, ferme toujours tes balises, et organise ton projet dans des dossiers (`assets/css`, `assets/js`, `assets/img`).

Réf. Écriture propre du HTML - MDN.

VIII-5 Les outils développeur des navigateurs

Chaque navigateur moderne (Chrome, Firefox, Edge, Safari) intègre des **outils de développement** accessibles par clic droit → *Inspecter* ou via la touche **F12**.

- **Inspecteur** : affiche le DOM en temps réel et les styles appliqués. Nous pouvons modifier le HTML/CSS à la volée pour tester.
- **Console** : montre les erreurs JavaScript et permet de tester du code directement avec `console.log()`.
- **Onglet Réseau** : analyse les requêtes HTTP, leurs codes de réponse (200, 404, 500…) et les temps de chargement.
- **Performances / Lighthouse** : mesure le temps de rendu, l'accessibilité, le SEO, et propose des recommandations.

Réf. Outils de développement Firefox - MDN.

VIII-6 Sources fiables de documentation

Pour progresser, il faut savoir où chercher :

- **MDN Web Docs** (developer.mozilla.org) : la référence la plus claire et la plus fiable pour HTML, CSS, JavaScript.
- **WHATWG HTML Living Standard** : norme HTML en constante évolution (html.spec.whatwg.org).
- **W3C** : versions publiées et archivées (w3.org/TR/html).
- **Can I use** (caniuse.com) : pour vérifier la compatibilité des fonctionnalités web.
- **Web.dev** (web.dev) : site de Google avec conseils et audits de performance.

Attention : certains sites comme *W3Schools* sont pratiques pour débuter, mais pas toujours rigoureux. Vérifie toujours avec MDN.

VIII-7 Fiche-mémo

- **Validation** : validator.w3.org pour HTML, jigsaw.w3.org/css-validator pour CSS.
- **Accessibilité** : toujours un `alt`, des `<label>`, utiliser `aria-*` pour les rôles/états.
- **Évite** : `<font>`, `<center>`, styles inline, `onclick="…"`.
- **Sépare** : HTML = contenu, CSS = style, JS = logique.
- **Outils navigateur** : Inspecteur, Console, Réseau, Lighthouse.
- **Docs fiables** : MDN, WHATWG, W3C, Can I use, web.dev.